1. Model Training

In this project, I developed a Python-based application that can analyze images and video streams to determine whether waste is correctly sorted. The system focuses on three types of objects: beverage cartons, deposit cans, and food cans. The goal was to combine object detection with a simple distance-based logic to visually indicate whether the sorting is correct or not.

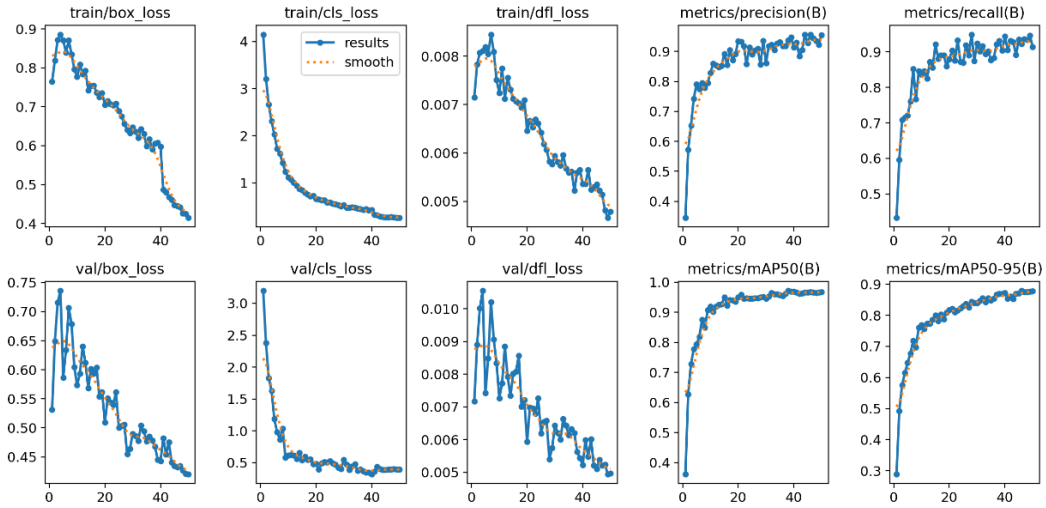
The work began with the creation of an annotated dataset. Each participant contributed at least 30 images containing the three classes. The images were annotated in Label Studio, where each object was marked with bounding boxes and assigned the correct class. The dataset was then exported in YOLO format, meaning that each image has a corresponding text file with coordinates and class information. To ensure consistency, a shared class definition was used in the following order: beverage carton, food can, and deposit can.

I first imported the images into Label Studio to verify that all annotations were correct. The dataset was then exported in YOLO format to a personal drive and imported into Google Colab. After compiling the dataset, I split it into three parts: training data (80%), validation data (10%), and test data (10%). This split enables both effective training and fair evaluation of the model's performance. The process was carried out automatically in Google Colab using Python, since the dataset was too large to manage manually.

The model I used was a YOLO model from the Ultralytics library. Training was performed in Google Colab using a T4 GPU, which provided significantly better performance compared to CPU. The model was trained for 50 epochs with an image size of 640 pixels and a confidence threshold of 0.4. During training, both loss values and mAP (Mean Average Precision) were logged, allowing analysis of the model's progress over time.

The training results showed that the model achieved very strong performance. The mAP50 value was approximately 0.988 and the mAP50-95 value around 0.891, indicating high accuracy in object detection. Precision was about 0.954 and recall about 0.967, meaning the model detects most objects while keeping misclassifications low.

Figure 1 shows the development of loss values and performance metrics during training. The loss decreases steadily, while precision, recall, and mAP increase and stabilize at high levels, indicating that the model converges well and generalizes effectively.



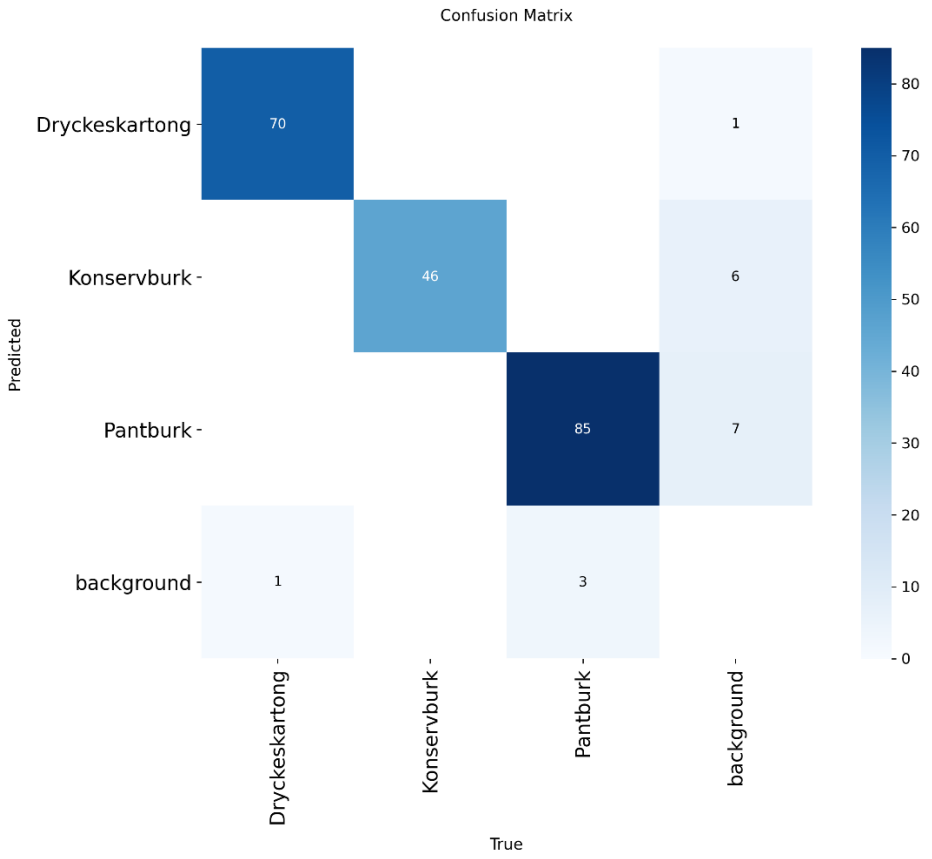
2. Evaluation and Analysis

To evaluate the model's performance, I conducted a quantitative analysis on the reserved test dataset, which represented approximately 10% of the total data. The results show that the model performs very well, with an mAP50 value of about 0.988 and an mAP50-95 value of about 0.891. This indicates high detection accuracy even under stricter overlap requirements between predictions and ground truth.

The precision was measured at approximately 0.954 and recall at approximately 0.967, indicating that the model successfully identifies a large proportion of objects while keeping false detections low. This demonstrates a good balance between detecting objects and avoiding false positives.

The confusion matrix (Figure 2) shows how the model's predictions are distributed across the different classes. Most objects are correctly classified, confirming the model's strong performance. A small number of misclassifications occur, mainly between food cans and deposit cans, which is expected due to their similar shape and appearance.

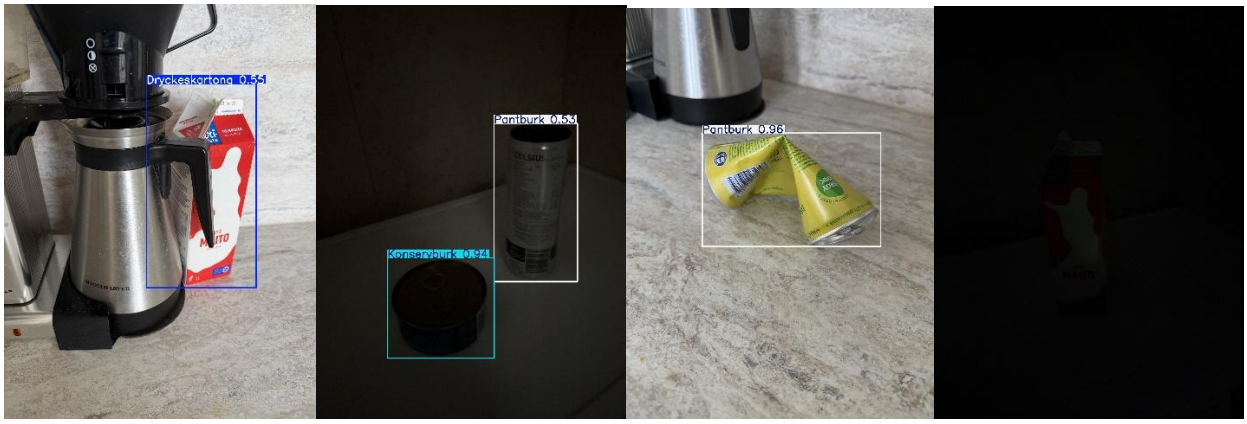
Overall, the quantitative results provide a reliable and accurate representation of the model's performance and show that it is well trained for the task.



To further evaluate the model's robustness, I conducted a so-called "stress test." In this test, I used ten new images that the model had not previously seen. These images were intentionally challenging, for example with poor lighting conditions, partially occluded objects, and varying backgrounds.

The results of the stress test showed that the model was still able to correctly identify objects in most cases. However, it struggled with very dark images, where two out of ten images produced no detections at all. This suggests that the model is sensitive to lighting conditions and that its performance decreases with low image quality.

Compared to the standard test dataset, the results from the stress test are slightly worse, which is expected since the images are more extreme and not fully representative of the training data. Despite this, the model demonstrates good generalization ability.



3. Application Implementation

After training and evaluating the model, I implemented it in an application for real-time object detection and sorting. The application was developed in Python using the Ultralytics YOLO library.

The trained model was loaded into the program and used to analyze images and video streams. For each frame, the model identified the object class, position, and associated confidence score. Based on the detected bounding boxes, I calculated the centroids of the objects, making it possible to determine their position in the image.

To enable sorting, I divided the image into different regions, where each region corresponded to a specific object category. Based on the object's class and its position, the system determined whether the object was correctly sorted or not.

The results were visualized directly in the image by drawing bounding boxes around the objects. The color of the boxes indicated the status: green for correctly sorted objects and red for incorrectly placed objects. This provided clear and intuitive real-time feedback.

The system was tested on both individual images and video data, showing that it works reliably and can be used for simple visual sorting tasks.